# NURobotics Firmware Course

Week 3: Serial Communication Protocols

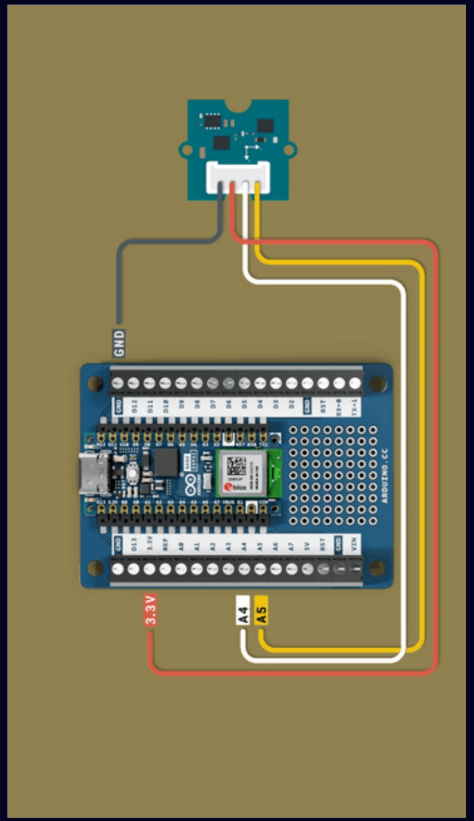
# Course Overview

- Week 1: Intro to Firmware & Programming
- Week 2: Analog I/O & Interrupts
- **Week 3: Serial Communication Protocols**
- Week 4: Driver Development
- Week 5: State Machines + RTOSes
- Week 6: STM32 Fundamentals
- Week 7: STM32 Peripherals
- Week 8: Communication and RTOS Capstone

# Serial Communication Protocol Background

# What Are Serial Communication Protocols?

- Devices in embedded systems need some way to communicate with each other
- Communication protocols are a set of agreed-upon rules for how devices exchange data (kind of like a language)
  - **Physical Connections:** *How many wires?*
  - **Data Rate:** *How fast is data transmitted?*
  - **Timing:** *When can devices transmit? How are signals synchronized?*
  - **Addressing:** *How does a device identify who should receive the data?*
- Different protocols make trade-offs between speed, complexity, cost, distance, reliability, etc.
- Serial communication protocols transmit data 1 bit at a time (hence the serial prefix)



# Logic Levels

- Logic levels generally refer to the voltage range of a signal
  - What voltage is HIGH and what voltage is LOW
  - Logic levels have threshold values for determining if a signal is HIGH or LOW
- Common logic levels
  - 5V (Arduino, older Arduino-compatible sensors)
  - 3.3V (ESP32, Raspberry Pi, many sensors)
- DON’T CONNECT A 5V OUTPUT TO A 3.3V INPUT!!!
- Logic level converters exist if needed

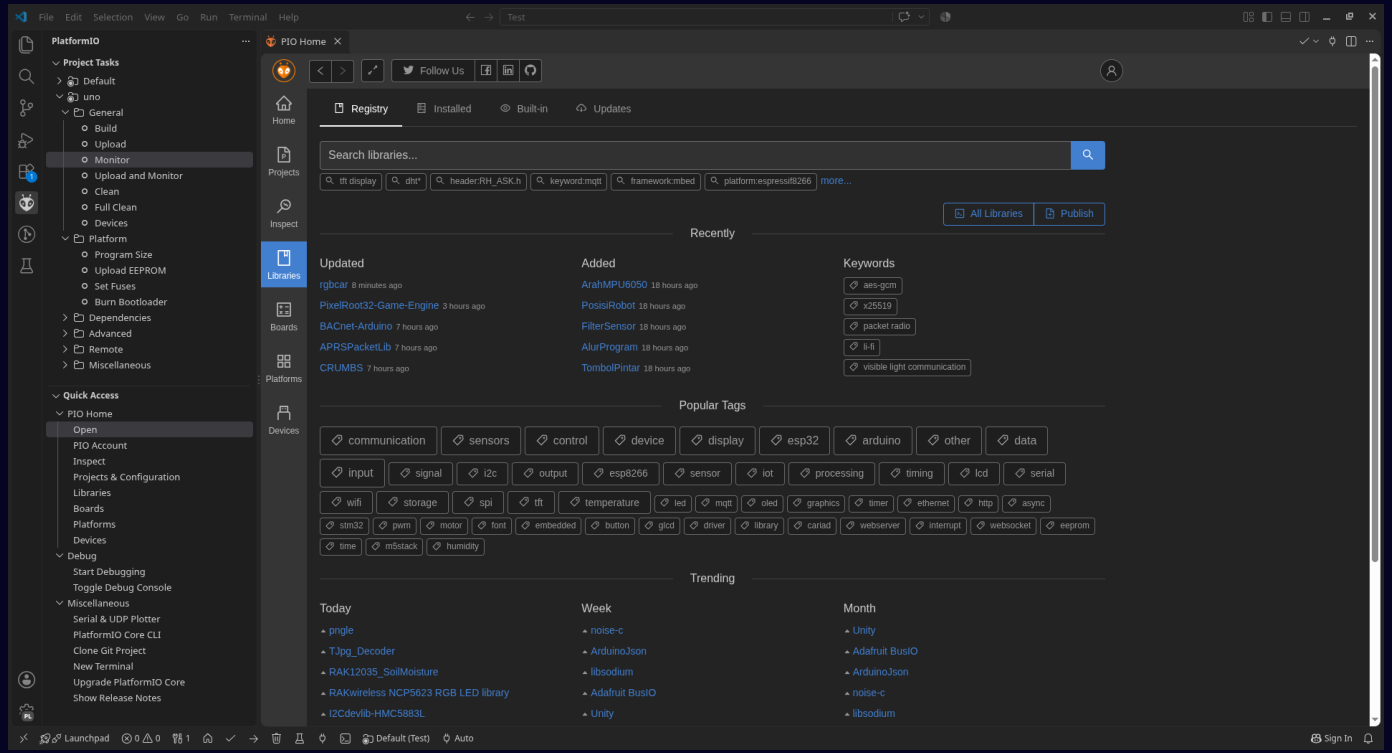
# Terminology

- Master / Controller – The device initiating the communication and controlling communication flow
- Slave / Peripheral – The device responding to the communication request
- Half-Duplex – Communication is possible both directions, but only one direction at a time, not simultaneously in both directions
- Full-Duplex - Communication is possible simultaneously in both directions

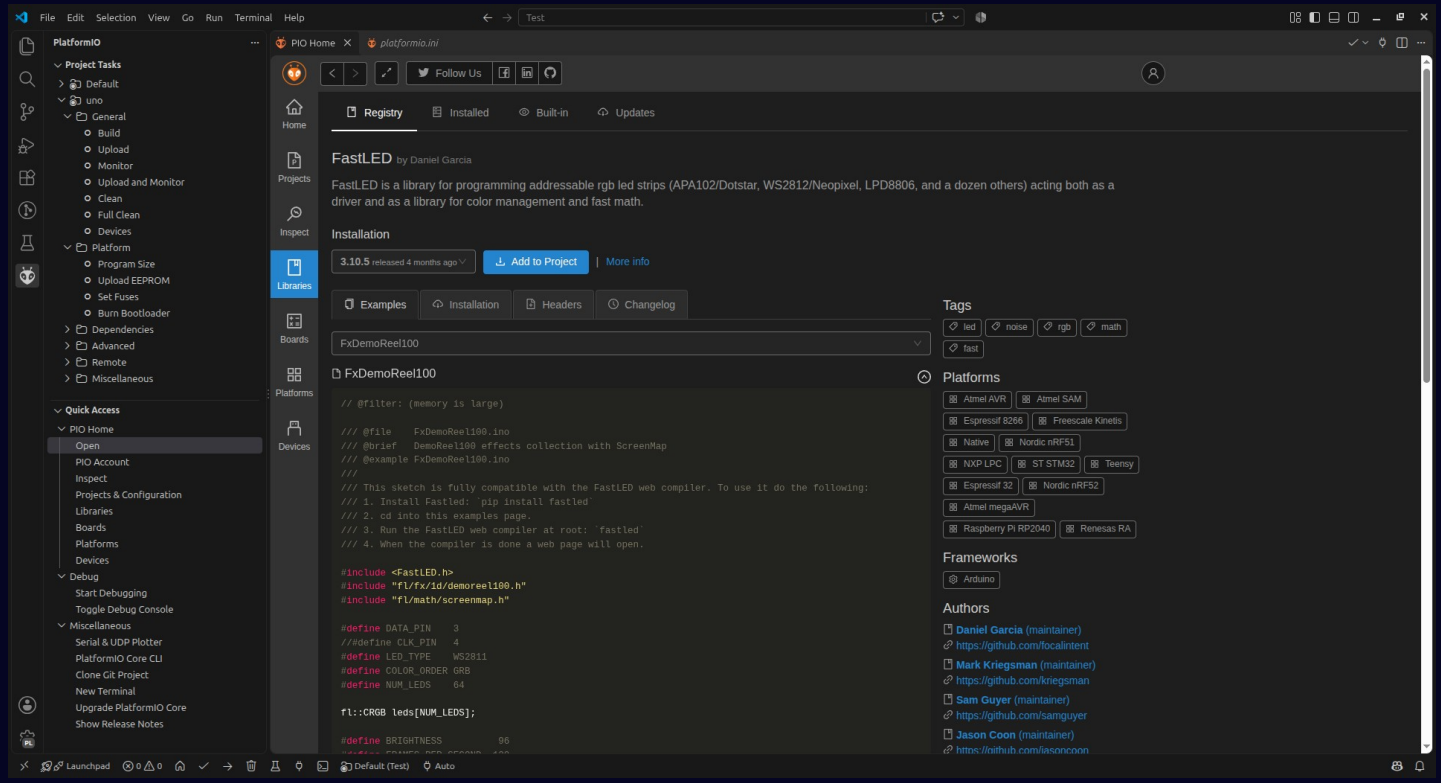
# Libraries

# What are Libraries?

- Collections of pre-written, reusable code that provide specific functionality



# Library Example



# Header Files

- C/C++'s solution to separating code into multiple files while allowing resources from one file to be pulled into another file
- Two types of files in C/C++:

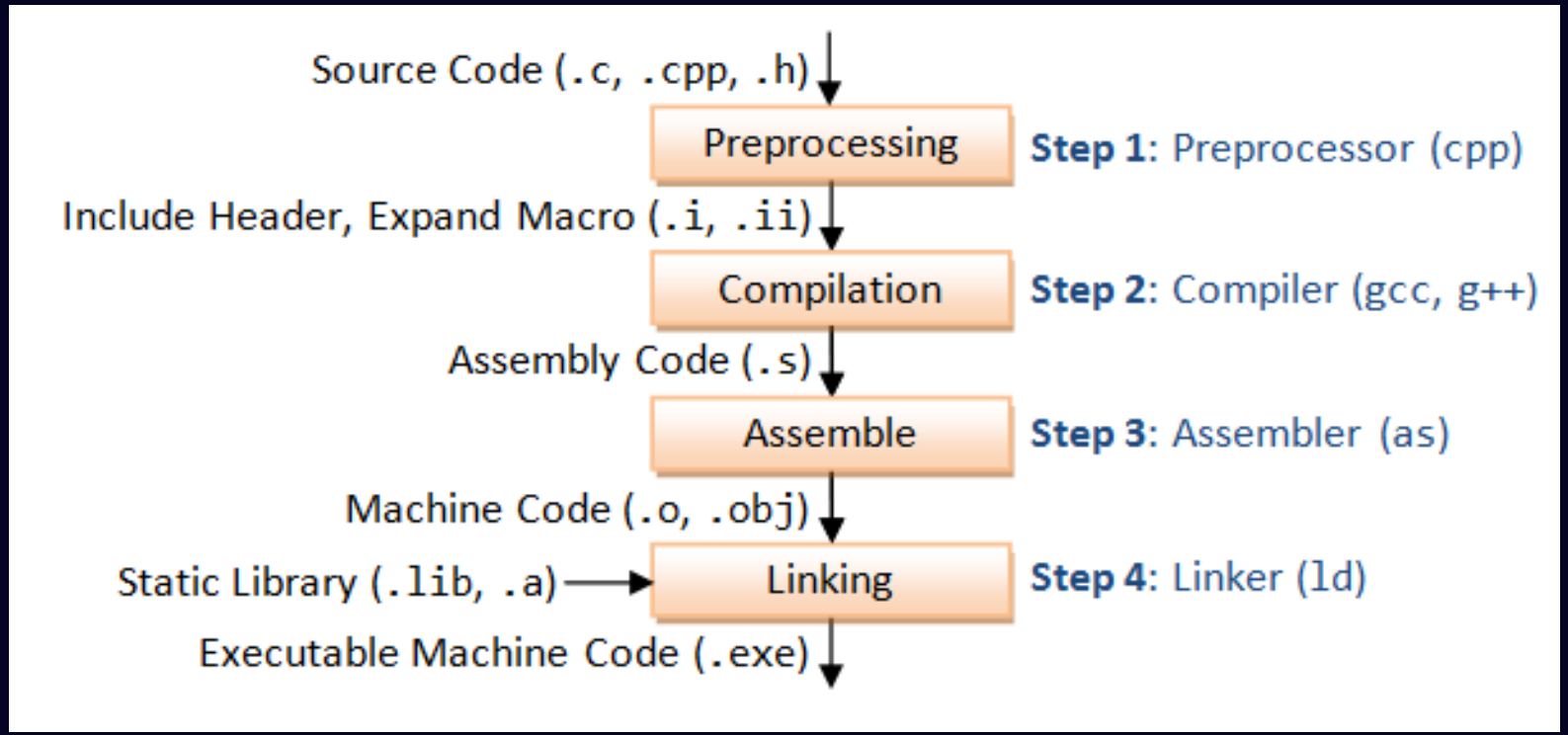
**Header Files:**
- *.h or *.hpp
- Declare symbols
  - Classes/functions/global variables

```
Void Func();
```

**Implementation Files:**
- *.c or *.cpp
- Define symbols
  - Function definitions
  - Global variable initializers

```
Void Func() {
    // implementation
}
```

# Compilation Steps



# Including Libraries

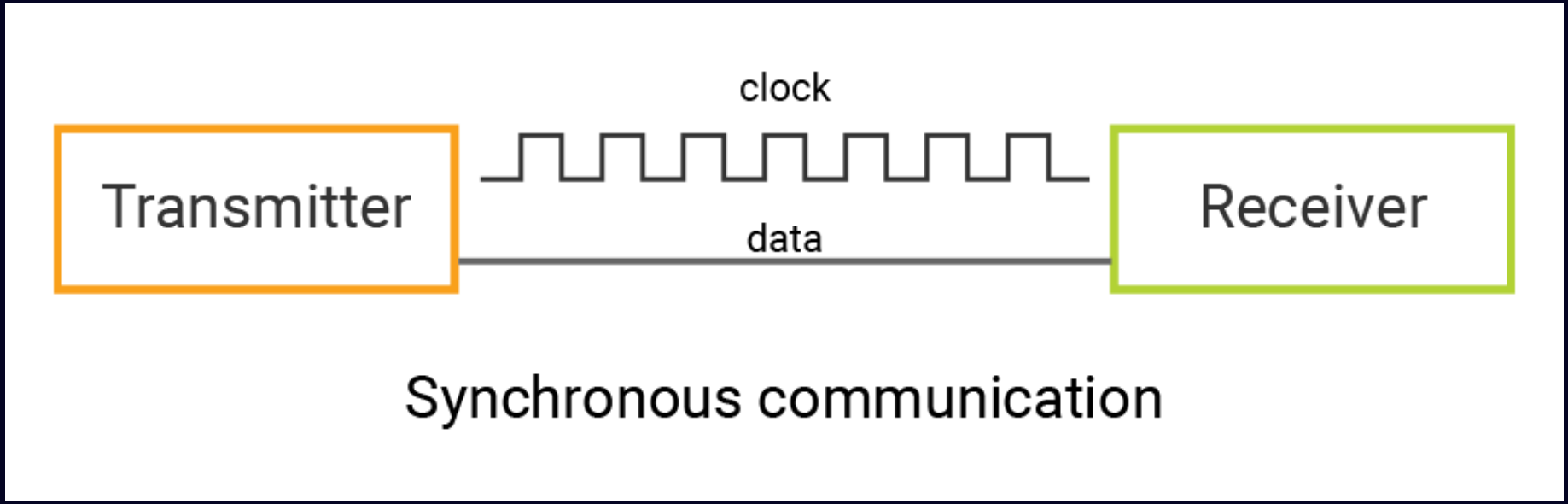
```
#include <Arduino.h>
#include <FastLED.h>

#include "test_header.h"
```

# Synchronous Communication

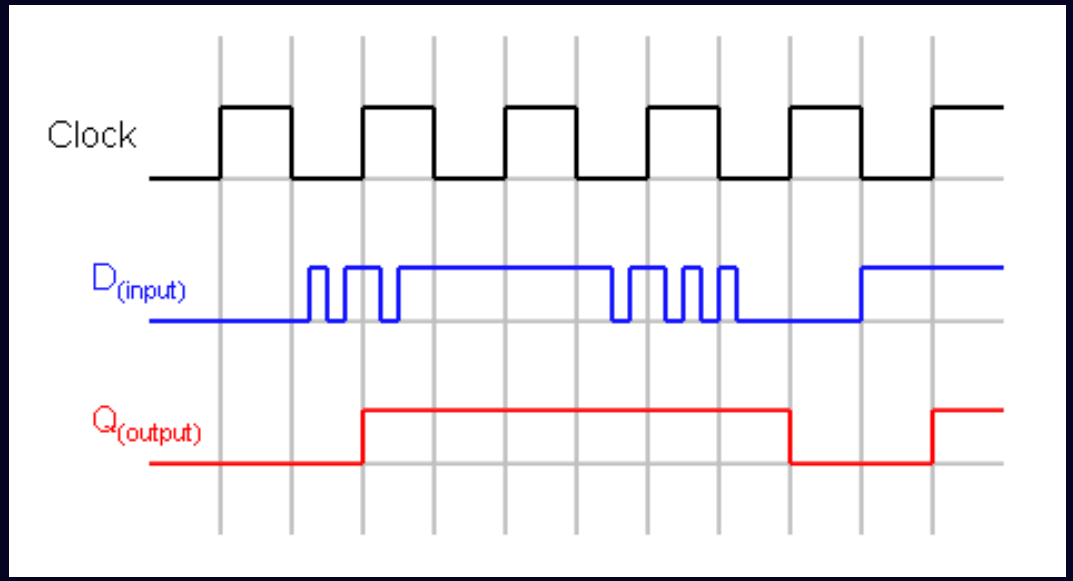
# Synchronous Communication

- Relies on a shared clock line between transmitter and receiver



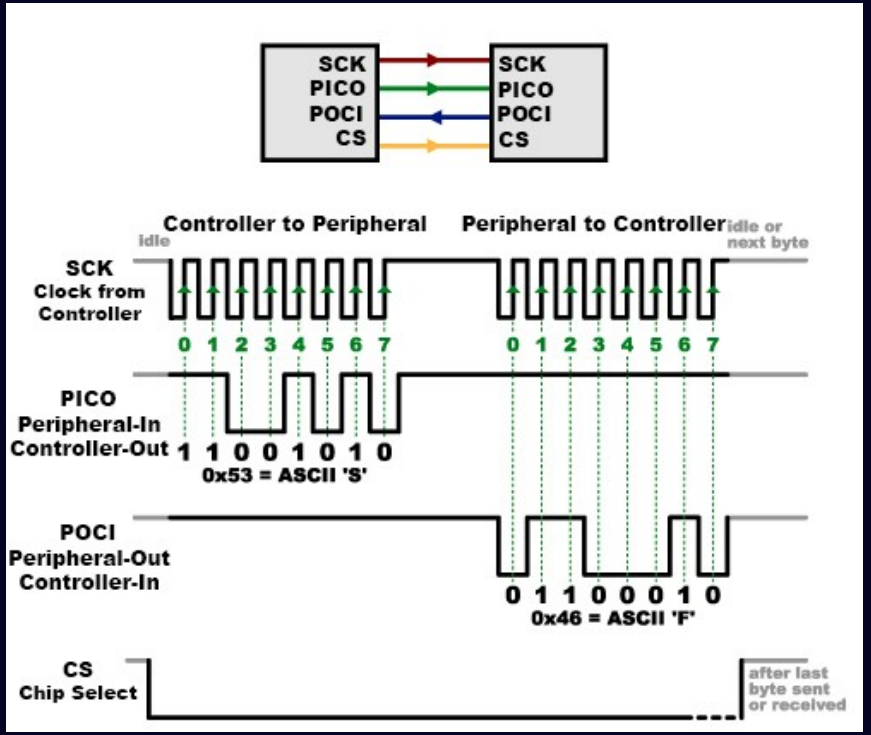
# Clocks

- Synchronize when data is read/written
  - Usually on the rising edge
- Clock speed (frequency) determines how fast bits are transferred

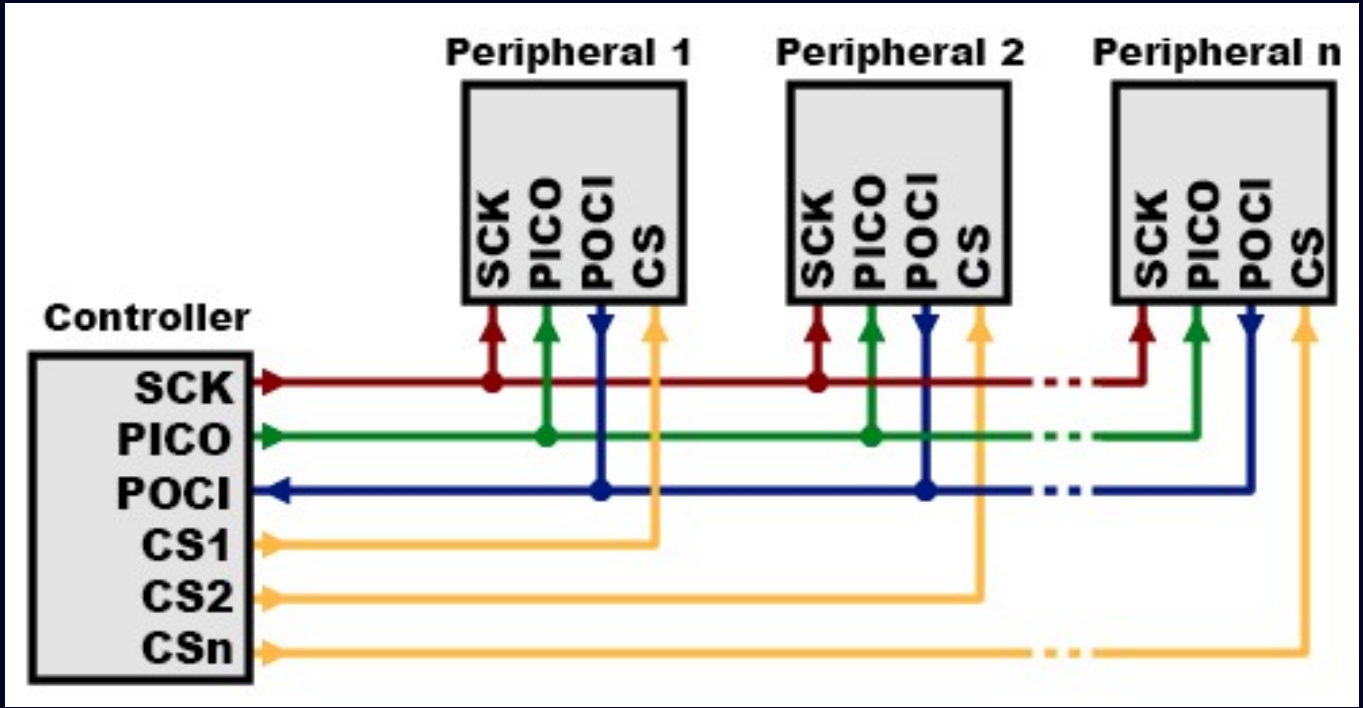


# SPI (Serial Peripheral Interface)

- One main and multiple peripheral devices
- 4 Wires
  - SCLK
    - Clock
  - MOSI/PICO
    - Master Out Slave In / Peripheral In Controller Out
  - MISO/POCI
    - Master In Slave Out / Peripheral Out Controller In
  - SS/CS
    - Slave Select / Chip Select
- More common for high speed sensors



# SPI: Single Controller – Multiple Peripherals

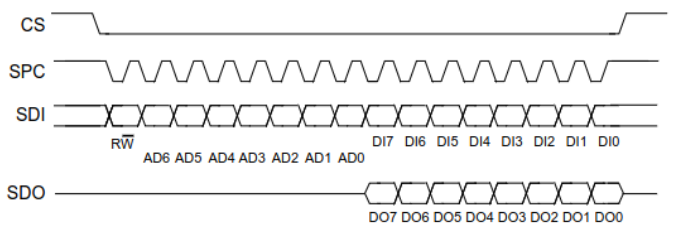


# SPI Example

### 5.1.3 SPI bus interface

The SPI on the LSM6DSV16BX is a bus slave which allows writing and reading the registers of the device.

Figure 11. **Read and write protocol (in mode 3)**




**CS** enables the serial port and it is controlled by the SPI master. It goes low at the start of the transmission and goes back high at the end. **SPC** is the serial port clock and it is controlled by the SPI master. It is stopped high when **CS** is high (no transmission). **SDI** and **SDO** are, respectively, the serial port data input and output. Those lines are driven at the falling edge of **SPC** and should be captured at the rising edge of **SPC**.

Both the read register and write register commands are completed in 16 clock pulses or in multiples of 8 in case of multiple read/write bytes. Bit duration is the time between two falling edges of **SPC**. The first bit (bit 0) starts at the first falling edge of **SPC** after the falling edge of **CS** while the last bit (bit 15, bit 23, ...) starts at the last falling edge of SPC just before the rising edge of **CS**.

**bit 0**: R$\overline{W}$ bit. When 0, the data DI(7:0) is written into the device. When 1, the data DO(7:0) from the device is read. In latter case, the chip will drive **SDO** at the start of bit 8.

**bit 1-7**: address AD(6:0). This is the address field of the indexed register.

**bit 8-15**: data DI(7:0) (write mode). This is the data that is written into the device (MSb first).

**bit 8-15**: data DO(7:0) (read mode). This is the data that is read from the device (MSb first).

In multiple read/write commands further blocks of 8 clock periods will be added. When the CTRL3 (12h) (IF_INC) bit is 0, the address used to read/write data remains the same for every block. When the CTRL3 (12h) (IF_INC) bit is 1, the address used to read/write data is increased at every block.

The function and the behavior of **SDI** and **SDO** remain unchanged.

# SPI Arduino Example

```ino
#include <Arduino.h>
#include <SPI.h>

#define CIPO 12
#define COPI 11
#define SCLK 13
#define CSN 10

void setup() {
  pinMode(COPI, OUTPUT);
  pinMode(CIPO, INPUT);
  pinMode(SCLK, OUTPUT);
  pinMode(CSN, OUTPUT);
  digitalWrite(CSN, HIGH);
  SPI.begin();
}

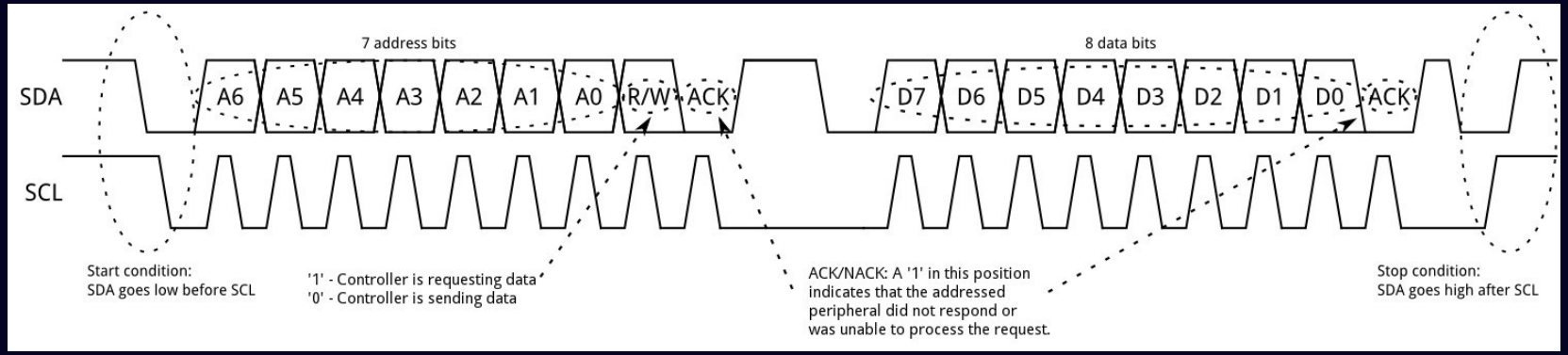
void loop() {
  // Start SPI transaction at 1 Mhz
  SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));

  // Transfer (Full-Duplex) over SPI peripheral
  uint8_t received = SPI.transfer(100);

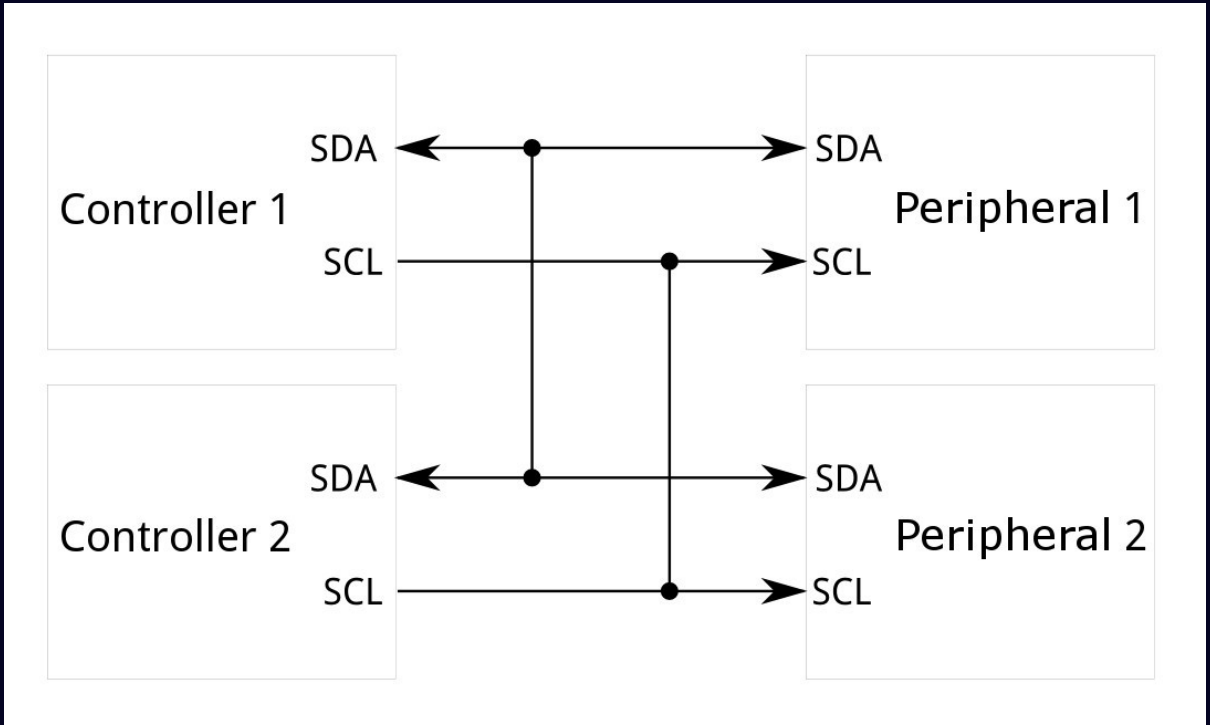
  // Stop SPI transaction
  SPI.endTransaction();
}
```

# I²C (Inter-Integrated Circuit)

- Multiple main and peripheral devices possible on the same bus
- 2 Wires
  - SDA
    - Data
  - SCL
    - Clock
- More common for low speed sensors



# I²C: Multiple Controllers – Multiple Peripherals



# I²C Example

## READING DATA

Reading data from the ADT7420 is done in a single data byte operation for the configuration register, the status register, the $T_{HYST}$ setpoint register, and the ID register. A two data byte read operation is needed for the temperature value register, $T_{HIGH}$ setpoint register, $T_{LOW}$ setpoint register, and the $T_{CRIT}$ setpoint register. Reading back the contents of an 8-bit register similar to the configuration register is shown in Figure 16. Reading back the contents of the temperature value register is shown in Figure 17.

Reading back from any register first requires a single-byte write operation to the address pointer register to set up the address of the register that is going to be read from. In the case of reading back from the 2-byte registers, the address pointer automatically increments from the MSB register address to the LSB register address.

To read from another register, execute another write to the address pointer register to set up the relevant register address. Thus, block reads are not possible, that is, there is no I²C address pointer auto-increment except when reading back from a 16-bit register. If the address pointer register has previously been set up with the address of the register that is going to receive a read command, there is no need to repeat a write operation to set up the register address again.

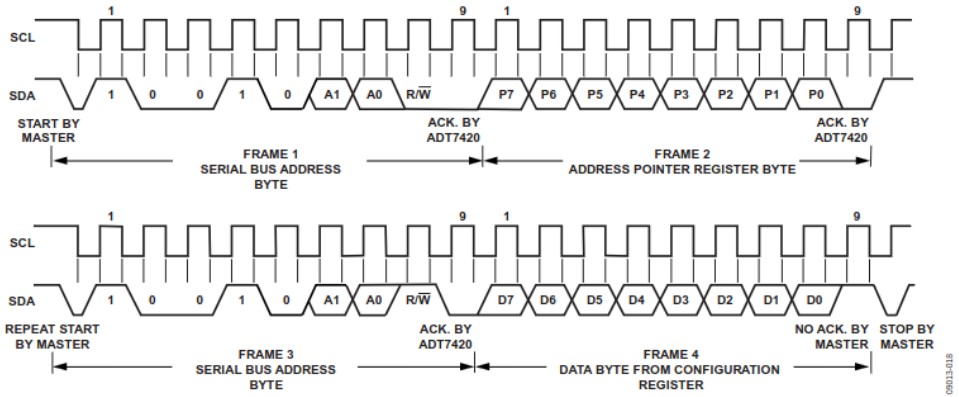



*Figure 16. Reading Back Data from the Configuration Register*

# I²C Arduino Example

```ino
#include <Arduino.h>
#include <Wire.h>

void setup() {
  Wire.begin();
}

void loop() {
  // Request 6 bytes from device ID 8
  Wire.requestFrom(8, 6);

  while (Wire.available()) {
    uint8_t data = Wire.read();
  }

  // Starts an I2C transmission to device ID 8
  Wire.beginTransmission(8);

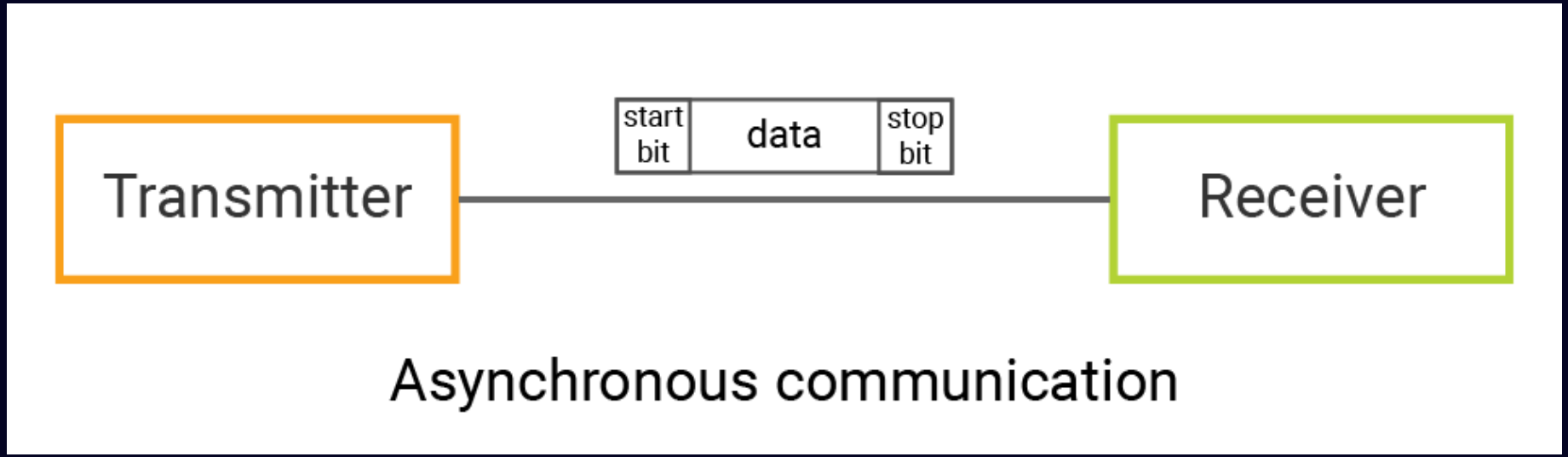
  // Adds the characters 't', 'e', 's', and 't' to the transmission buffer
  Wire.write("test");

  // Sends the data in the transmission buffer
  Wire.endTransmission();
}
```

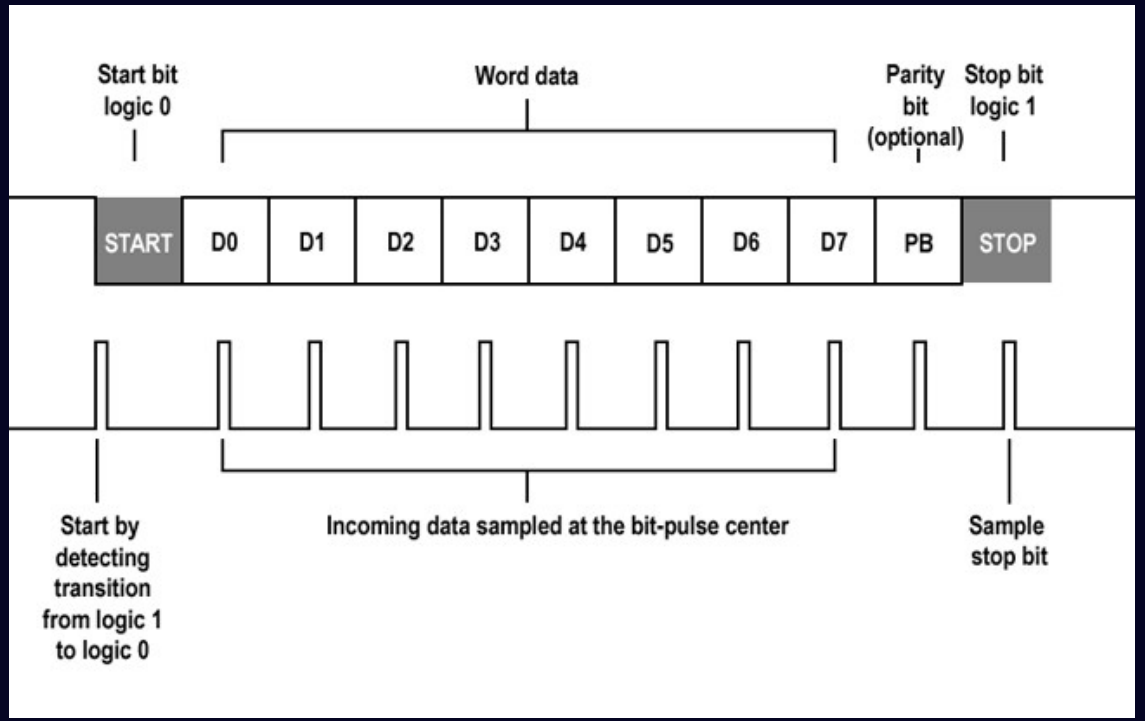
# Asynchronous Communication

# Asynchronous Communication

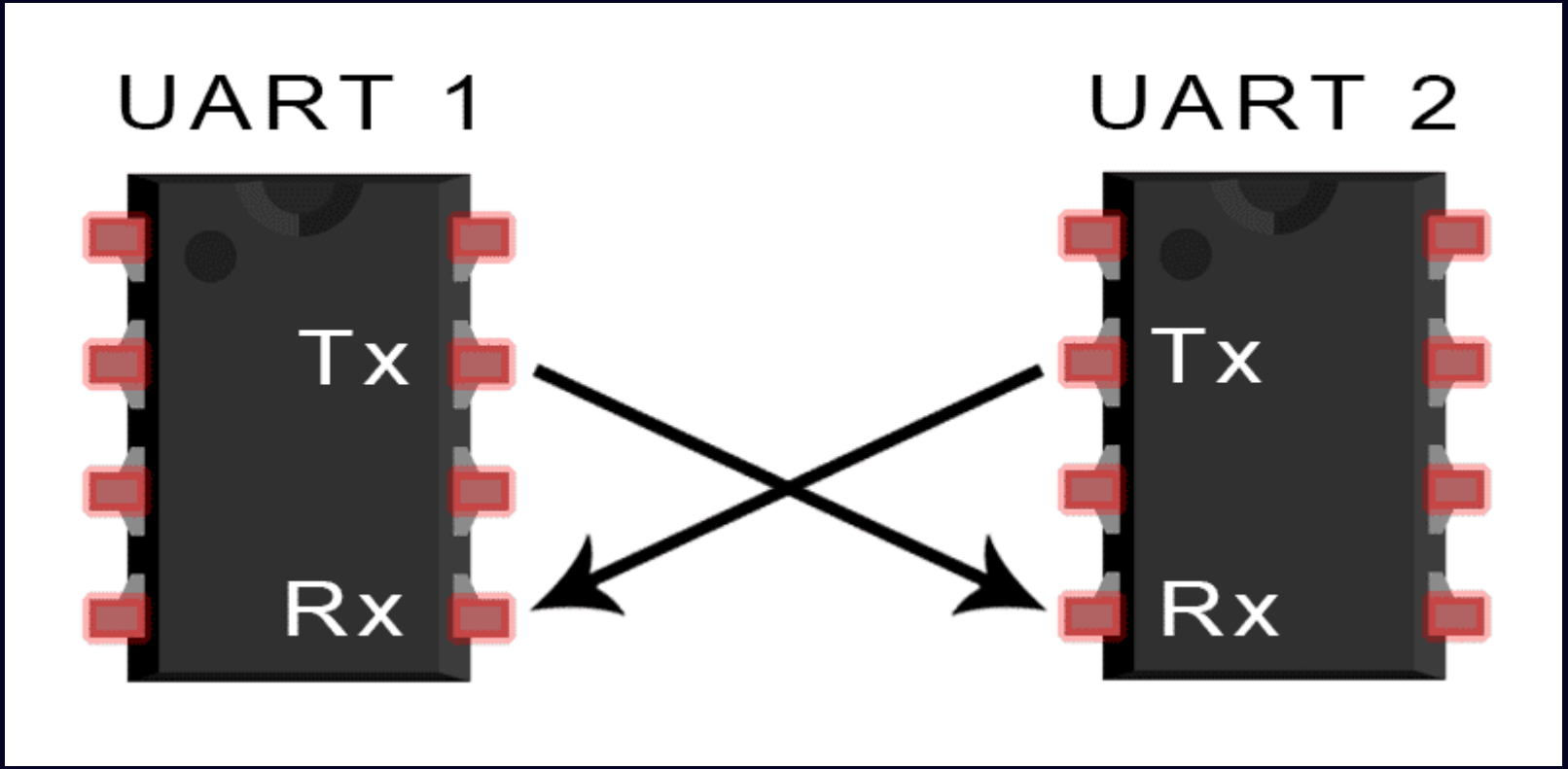
- No shared clock signal between devices
- Devices agree on a data data rate ahead of time (baud rate)
  - Usually uses start and stop bits to indicate beginning and end of each data frame



# UART Example



# UART Lined (TX/RX)



# UART Arduino Example

```ino
#include <Arduino.h>

void setup() {
    // Start the USB serial at 9600 baud
  Serial.begin(9600);
}

void loop() {
  // Send data over the USB serial port
  Serial.println("Hello World");

  // Receive data over the USB serial port
  if (Serial.available() > 0) {
    uint8_t data = Serial.read();
  }
}
```

# Serial Communication Comparison

| | SPI | I²C | Serial (async) |
|---|---|---|---|
| **Typical Speed** | Multiple Mbps | 400 kbps | 115200 bps |
| **Typical Usage** | IC to IC | IC to IC, device to device | Device to device |
| **Number of Devices** | 2 to many | 2 to 127 | 2 only |
| **Number of Comm Wires** | (n – 1) + 3 | 2 | 2 to 6 or more |
| **Polarity** | + | + | + and - |
| **Signaling** | 0-1.8V to 0-5V | 0-1.8V to 0-5V | 0-3.3V up to 15 to -15V |
| **Pin Mode** | Regular drive | Open collector | Regular drive |
| **External Components** | None | 2x 4.7K resistor | Maybe level converter |
| **Distance** | Inches | Feet | Yards |

# Lab!

# Today’s Lab

- Controlling Arduino with the Serial Monitor
  - Send commands from the serial monitor to change the PWM of LEDs
- IMU LED toggle
  - Toggle LEDs on and off by shaking the IMU